

# LocAgent: A Semi-Agentic System For Software Issue Localization

Anonymous ACL submission

## Abstract

pass

## 1 Introduction

Repository-level code generation tackles the task of, given an issue or feature description and a code-base/repository, proposing a patch in the correct location to fix the issue or implement the feature. Existing systems (Xia et al., 2024; Wang et al., 2025; Yang et al., 2024) typically follow a 3-stage pipeline: a **localizer** to identify the affected code, a **patch generator** to propose a fix, and a **verifier** to check correctness. Trivially the performance of each of these components is bounded by the performance of the previous component, i.e. for patch generator to be successfully the localizer needs to provide the correct spots of the edit. Agentless (Jimenez et al., 2024a) achieves 70% file-level localization (lower function and line-level) correctness using GPT-4o on SWE-Bench Lite (Jimenez et al., 2024b) and 32% overall resolve rate. To maximize the localization systems performance we focus on the localization component in the scope of this work, later on evaluating our improved localization system in an end to end setup to show the effectiveness.

Issue localization the task of pinpointing the code that must change to fix a software problem is essential yet notoriously difficult. The gap between natural-language descriptions and the underlying issue requires multi-hop reasoning over code and its dependencies. Current LLM-based agents address this by augmenting themselves with repository retrieval and inspection tools. In general localization systems are expected to provide fine grained locations of edit e.g. file-path(s) in the repository and the number of lines of the chunk(s). Previous works vary on the level of granularity of the provided locations, such as SweRank (Reddy et al., 2025) performs function-level localization,

nevertheless not all patches are within function changes, e.g. a change in the imports is required. There are essentially 2 core approaches to issue localization. In this work we also try to answer to the question of whether we need to pin-point the exact line numbers or providing the files is already enough. Moreover providing a narrow chunk/window in a file might not provide enough context to the patch generator model about the file/class dependencies.

**Retrieval Based** methods using encoder-only models to extract a latent representation of code chunks in the repo and match them with the issue description based on vector similarity metrics. This method is limited due to the earlier mentioned gap between natural-language description and code-chunks. Moreover it multiplicatively increases the overall cost of repository-level codegen tasks (e.g. SWE-Bench Jimenez et al. (2024a)), since the output of the localization step should be fed to the patch generator and the output of the later to the verifier module.

**LLM-based** where an agentic model is meant to perform various tool calling (e.g., searching specific function by keyword, listing functions that call a certain function) operations to dynamically explore repositories. However, the latter approach places higher demands on LLMs: beyond multi-step reasoning, they require sophisticated tool-calling abilities to explore code effectively rather than walking aimlessly through the repository.

Agentic codegen systems such as OpenHands Wang et al. (2025) utilize agentic LLMs that can iteratively read code, call tools, and update their hypotheses abilities that are well suited to fault localization. It requires an agent (tool calling LLM) to write legit bash commands that will be executed in the repository dir. However often times an agent pipes multiple commands such as "ls", "find", "cat" and more together to get a representative information, which is not the case for human

developers when localizing a bug. Moreover these systems often times spend half of the overall interaction turns on localization task, essentially occupying unnecessary context for the patch generator component.

Given these shortcomings there is a need to have a specialized localization system that will work separately from the other components by simply providing the locations of the edit, leaving room for the other 2 components to actively iterate and refine the solution. **LocAgent** emulates agentic behavior to produce multi-turn localization trajectories at scale with open-weight models and a finite tool set, then distill a smaller student model to pinpoint buggy lines.

## 2 Related Work

OpenHands (Wang et al., 2025) simulates the repo exploration in a sandbox, it uses tools such as "open\_file", "goto\_line", "scroll\_down" that actually open the files and wait for an action in the next turn.

SWE-Agent (Yang et al., 2024) has specialized tools, including an interactive file viewer, search functionalities, and edit tools for the open file, in addition to the standard Linux Bash commands, similar to OpenHands(Wang et al., 2025).

Tool integrated reinforcement learning repo (Ma et al., 2025).

PathPilot (Li et al., 2025).

## 3 Method

We present LocAgent, a multi-turn interaction system designed for precise issue localization in software repositories. Unlike existing approaches that rely on unbounded bash commands or complex tool sets, LocAgent employs a focused set of four tools within a carefully managed dialogue framework to efficiently identify code locations requiring modification.

### 3.1 Multi-Turn Interaction Framework

LocAgent models issue localization as a sequential decision-making process where an autonomous agent iteratively explores the codebase through structured tool interactions. The agent operates within a dialogue framework, maintaining conversation history that captures both its reasoning process and information gathered from the repository.

The agent follows a perception-reasoning-action cycle: (1) receiving the current state comprising the problem description and accumulated tool outputs, (2) analyzing available information using a large language model to determine the next exploration action, and (3) executing one of four specialized tools to gather additional repository information.

### 3.2 Tool Design

We provide four carefully designed tools that enable comprehensive yet efficient repository exploration:

**File Viewing** retrieves source code content with optional line range specification, enabling focused examination of specific code sections while managing output size through configurable limits.

**Repository Structure** generates a hierarchical view of the codebase, intelligently filtered to show only relevant source files while excluding non-essential directories such as tests, build artifacts, and documentation.

**Code Search** performs case-insensitive keyword search across the repository, returning contextual snippets (20 lines before/after) around matches to facilitate understanding of code usage patterns. Results are ranked by match frequency and limited to maintain manageable output sizes.

**Dependency Analysis** constructs import relationship graphs to understand module interconnections, helping identify related components that may require modification.

### 3.3 Context Management

A critical challenge in multi-turn systems is managing growing dialogue history within language model context limitations. We developed a comprehensive context management strategy with multiple components:

#### 3.3.1 Token Estimation

We implement context-aware token estimation that differentiates between code and natural language. Code segments require approximately 3.5 characters per token due to operators and formatting, while natural language averages 4.5 characters per token. This differentiated estimation enables accurate context planning.

#### 3.3.2 Truncation Strategies

When dialogue history approaches context limits, we employ one of several strategies:

**Sequential Truncation** progressively removes oldest interaction turns while preserving recent context, suitable for problems requiring detailed tracking of recent explorations.

**Bookend Truncation** (default) retains the initial problem statement and most recent interactions while removing intermediate turns, ensuring the agent maintains problem awareness while focusing on current exploration state.

**Adaptive Truncation** dynamically selects between strategies based on conversation characteristics, with multi-level fallbacks for extreme cases requiring aggressive reduction.

### 3.3.3 Proactive Monitoring

Rather than reacting to context overflow errors, our system proactively monitors token usage before each interaction, calculating expected consumption and applying appropriate truncation strategies preemptively. A configurable safety margin (default: 90% of maximum context) ensures reliable operation.

## 3.4 Loop Detection and Intervention

Multi-turn systems risk entering repetitive behavior patterns. We address this through comprehensive loop detection and intervention:

The system continuously monitors tool usage patterns, maintaining a sliding window of recent actions. When consecutive identical tool calls exceed a threshold (default: 3), it identifies a behavioral loop and injects carefully crafted intervention messages suggesting alternative exploration strategies.

Beyond immediate intervention, the system logs repetitive patterns to identify common failure modes, informing both prompt engineering improvements and architectural enhancements.

## 3.5 Repository Filtering

Efficient exploration requires intelligent filtering to focus on relevant code:

**File Type Selection:** We restrict exploration to source code files most likely to contain implementation logic (Python and configuration files for Python repositories).

**Directory Filtering:** We exclude directories unlikely to contain production code including testing infrastructure, build artifacts, documentation, and third-party dependencies.

**Adaptive Preservation:** Based on empirical analysis, we preserve commonly excluded direc-

tories that frequently contain relevant code: utility modules, library directories, migration files, and configuration directories.

## 3.6 Final Turn Optimization

To maximize success rates given finite interaction budgets (default: 20 turns), we implement final turn optimization. When approaching the maximum allowed interactions, the system injects specialized instructions requiring the agent to synthesize accumulated information into concrete location predictions. This transforms potential failures due to interaction limits into partial successes based on accumulated evidence.

## 3.7 Robustness Features

Real-world deployment requires handling various failure modes gracefully:

**Format Flexibility:** The system accepts multiple tool invocation formats, including properly structured commands, partial specifications, and implicit requests, accommodating variations in language model outputs.

**Progressive Degradation:** When encountering errors, the system attempts multiple recovery strategies: suggesting similar files when exact matches aren't found, providing informative error messages with actionable suggestions, and maintaining operation despite individual tool failures.

**Implicit Tool Detection:** When agents fail to use proper tool call syntax, the system employs fallback strategies including JSON detection after reasoning blocks, simple file path extraction, and search query pattern matching.

## 3.8 Evaluation Framework

We implement comprehensive evaluation mechanisms including ground truth extraction from repository patches, multi-level metrics (file-level accuracy, line-level precision, coverage analysis), and systematic failure categorization to identify areas for improvement.

The modular design enables controlled experimentation while maintaining robustness across diverse repository structures and problem types, with all enhancements maintaining backward compatibility for ablation studies.

## References

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik

Narasimhan. 2024a. [Swe-bench: Can language models resolve real-world github issues?](#) *Preprint*, arXiv:2310.06770.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024b. [SWE-bench: Can language models resolve real-world github issues?](#) In *The Twelfth International Conference on Learning Representations*.

Hongwei Li, Yuheng Tang, Shiqi Wang, and Wenbo Guo. 2025. [Patchpilot: A cost-efficient software engineering agent with early attempts on formal verification](#). *Preprint*, arXiv:2502.02747.

Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. 2025. [Tool-integrated reinforcement learning for repo deep search](#). *Preprint*, arXiv:2508.03012.

Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025. [Swerank: Software issue localization with code ranking](#). *Preprint*, arXiv:2505.07849.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025. [Openhands: An open platform for ai software developers as generalist agents](#). *Preprint*, arXiv:2407.16741.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. [Agentless: Demystifying llm-based software engineering agents](#). *Preprint*, arXiv:2407.01489.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. [Swe-agent: Agent-computer interfaces enable automated software engineering](#). *Preprint*, arXiv:2405.15793.

## A Example Appendix

This is an appendix.